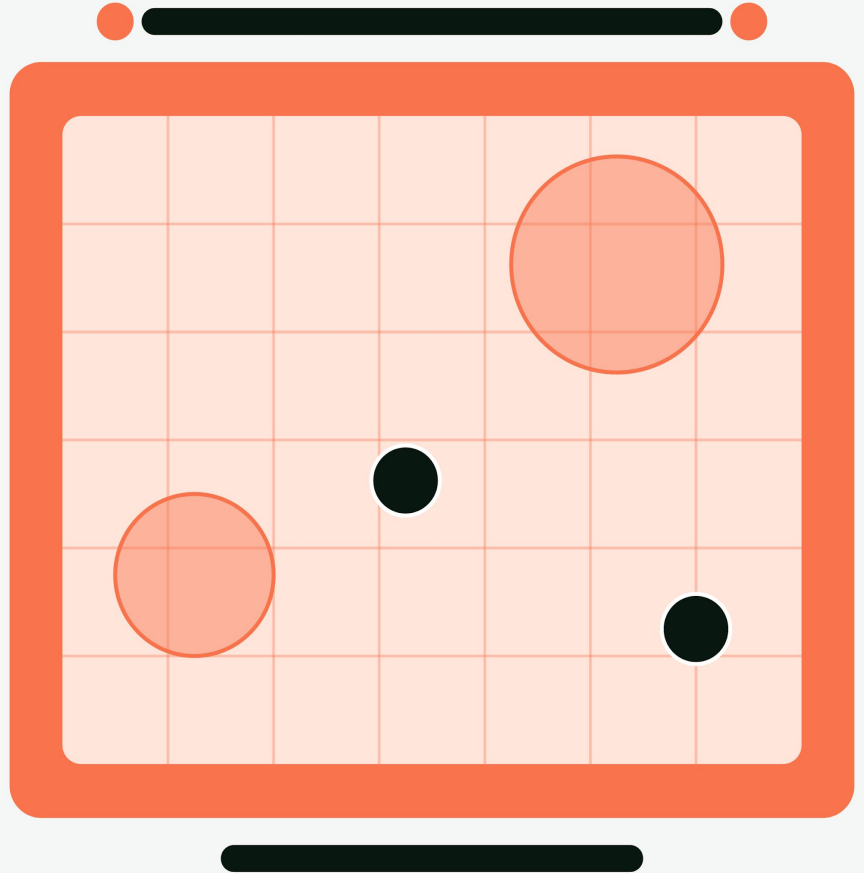


Front End. Practice 4

Agent Wars



Objective:

Build a custom realtime client for the Agent Wars Socket.IO server.

Basic Exercise:

- Download the sample client from the server homepage and open it in your browser.
- Restyle the UI: choose your own theme, layout and visual identity.
- Show every connected agent and the active incident zones in real time.
- Implement Phase 1 (manual move) and Phase 2 (claim & repair).
- Display your stats: zones resolved, time on board, score.

Advanced Exercise:

- Build your own client from scratch using socket.io-client and your favourite framework.
- Render the 1200×800 board with HTML, Canvas or SVG.
- Handle every event in the protocol (agent:join, agent:move, zone:claim, zone:*)).
- Build an autonomous agent (Phase 3): your code plays for you.
- Open a pull request to the agent-wars-server repository.

This base app can be extended in many ways — feel free to experiment and add features.

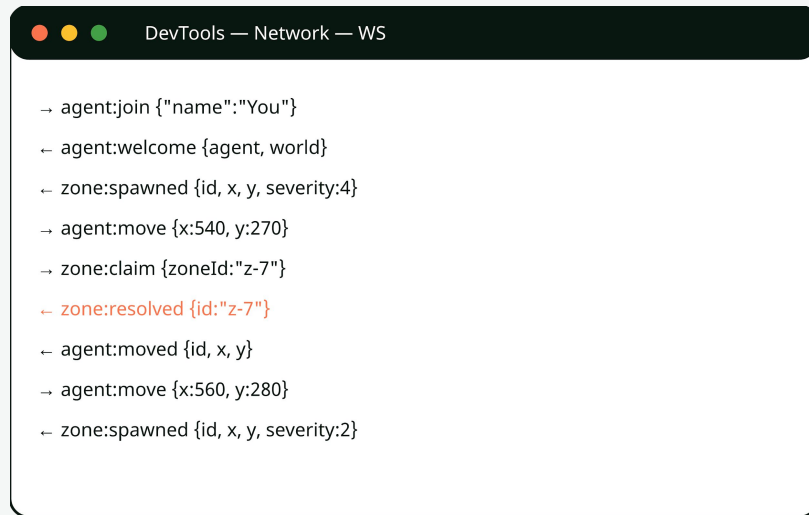
01. Understand the Protocol:

Read the Agent Wars homepage and identify the Socket.IO events you will send and receive.

You send: `agent:join`, `agent:move`, `zone:claim`

You receive: `agent:welcome`, `agent:joined`,
`agent:moved`, `agent:left`, `zone:spawned`,
`zone:updated`, `zone:resolved`

Board: 1200×800 px. Agents 40 px. Up to 5 active zones, severity 1–5, repaired in 2 s.



02. Connect & Join the Board:

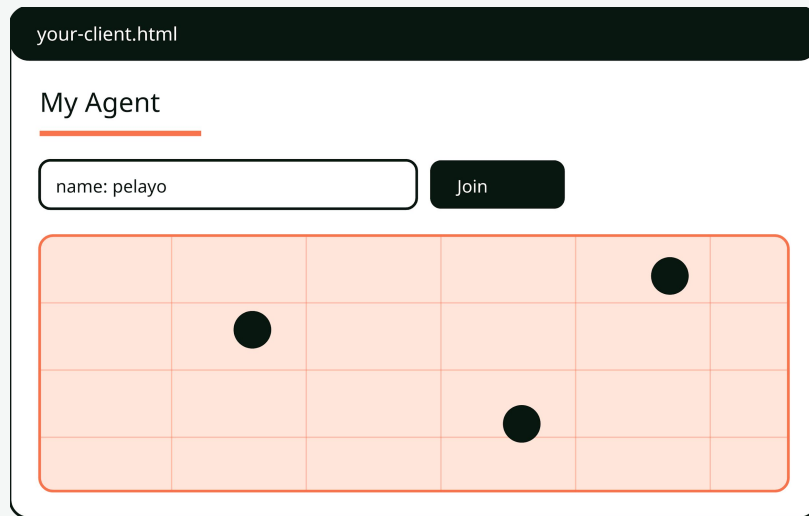
Phase 1 — get on the board and move your agent.

- Add the socket.io client (CDN or npm).
- Connect to the WSS endpoint of the server.

Emit `agent:join` with your name and read the welcome snapshot.

- Render every agent at their (x, y) coordinates.

Listen to `agent:joined` / `agent:moved` / `agent:left` and update the board.



03. Detect Zones & Repair:

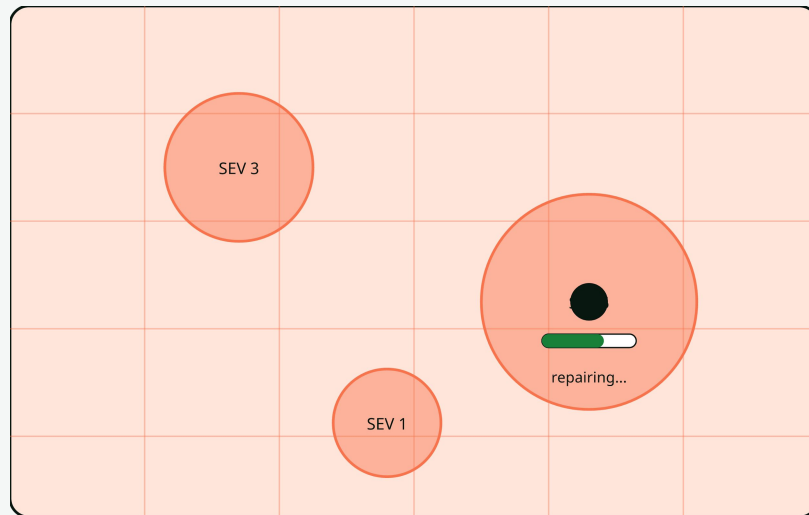
Phase 2 — paint the danger zones, race to claim them, hold position to repair.

Listen for `zone:spawned` and render the zone with its radius and severity.

When your agent enters a zone, emit `zone:claim` with the `zoneId`.

- Show a progress bar for the 2-second repair window.

On `zone:resolved`, remove the zone and increment your score.



04. Automate Your Agent:

Phase 3 — you stop playing. Your code plays for you.

- Detect every active zone from the world snapshot and live events.
- Pick a target zone (closest? highest severity? least contested?).

Navigate towards it: emit `agent:move` on a tick (e.g. 10 fps).

- Claim the zone when in range and hold position for 2 s.
- When resolved, choose the next target. Keep going.

`agent.loop()`

```
setInterval(() => {  
  const z = pickTarget(state.zones);  
  if (!z) return;  
  if (inRange(me, z)) {  
    socket.emit("zone:claim", z.id);  
  } else {  
    const next = stepToward(me, z);  
    socket.emit("agent:move", next);  
  }  
}, 100);
```

05. Submit a Pull Request:

Deliver your client through GitHub. Open a PR against the server repository:

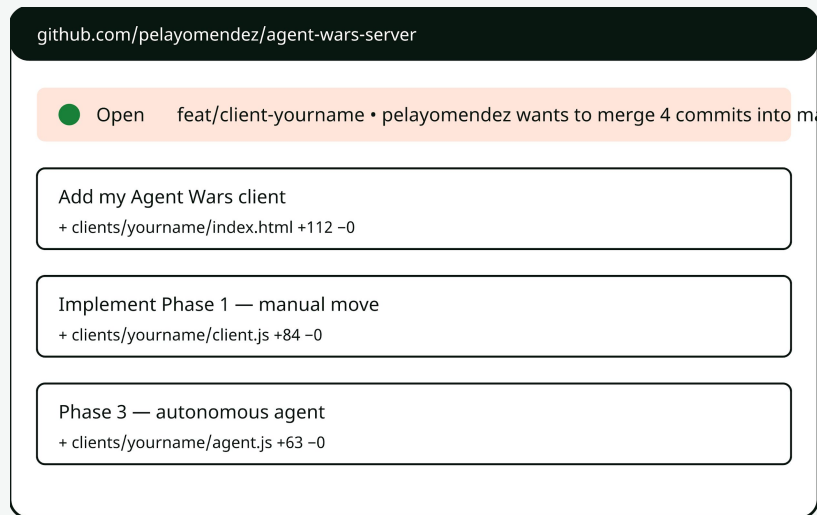
github.com/pelayomendez/agent-wars-server

- Fork the repo to your own GitHub account.

Add a folder `/clients/<your-name>/` for everything you build.

Branch from main: `git checkout -b feat/client-<your-name>`

- Commit your client (source, README and a screenshot).
- Push, open a PR with a clear title and demo gif, and reply to review comments.



Feature Ideas for Enhancements

UI & UX:

- Add a tactical mini-map and a permanent HUD with agent count and your repair score.
- Interpolate between agent:moved events instead of teleporting; add easing for a polished feel.

Strategy:

- Score zones by severity, distance and contention; try a greedy algorithm vs. a weighted one.
- If another agent is already claiming a zone, switch targets to maximise your throughput.

Data & AI:

- Track time-to-resolve, zones-per-minute and longest streak; visualise it with a small chart.
- Ask an AI API to choose the next target given the current world state; compare it to your heuristic.

Thank you for your time 😊